

Corso di Ingegneria del Software e Progettazione Web, limitatamente alla sezione a cura del
ricercatore Guglielmo De Angelis.

PREFAZIONE

Questi appunti affrontano alcuni argomenti fondamentali del corso e sono corredati di esempi di codice e di analogie. Il linguaggio di programmazione utilizzato sarà esclusivamente **Java**.

Contenuti esclusi: non verranno trattati temi quali semafori, thread, rappresentazioni numeriche, funzionamento interno della macchina, JVM, JDK/JRE. Si presuppone che l'intero stack software necessario sia già stato installato. Inoltre, molte istruzioni non verranno commentate, in quanto considerate conoscenze preliminari già acquisite. Non verranno altresì introdotti i concetti di database (per i quali si rimanda ai miei materiali sul corso di Basi di Dati), né le stored procedure, né i diagrammi E/R e le relative relazioni.

Obiettivo: l'attenzione sarà rivolta unicamente alla **progettazione del software**, non alla mera programmazione. È importante sottolineare che il valore non risiede nell'atto di "scrivere codice", bensì nel progettare con logica e metodo: ciò che distingue un buon risultato non è l'esecuzione meccanica, ma la capacità di collocare correttamente ogni elemento all'interno dell'architettura complessiva.



Il programma deve possedere una propria coerenza interna, che trae origine esclusivamente dal ragionamento del progettista. Occorre quindi conoscere prima la teoria, evitare di precipitarsi nella scrittura del codice, sviluppare gradualmente una **forma mentis** adeguata e solo successivamente cimentarsi nello sviluppo pratico.

Questi appunti sono stati redatti a seguito della mia frequenza al corso e con il supporto di alcuni materiali provenienti da un ex studente, Matteo Fanfarillo, i quali sono stati di grande supporto: i suoi appunti sono disponibili al seguente [link](#).

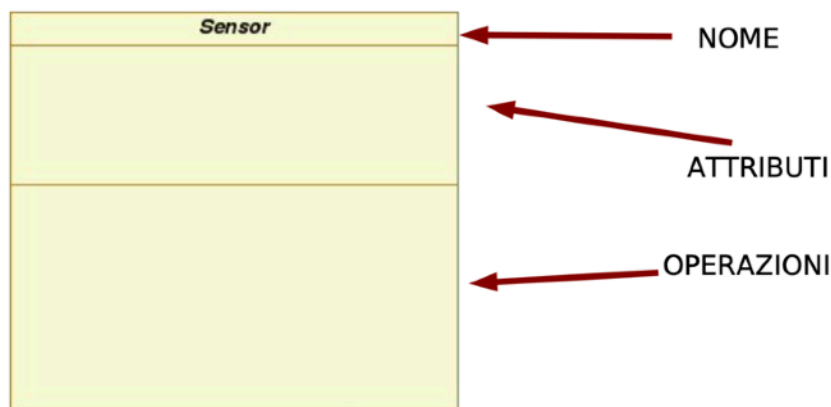
Buon inizio di studio.

Sarò molto diretto: cominciamo a trattare il tema della **progettazione**. Non è possibile affrontare questo argomento senza fare riferimento allo **standard UML** (Unified Modeling Language), che rappresenta il punto di partenza imprescindibile per una corretta modellazione del software. **UML** è un linguaggio visuale utilizzato per la specifica, la costruzione e la documentazione degli **artefatti** di un sistema software.

Si configura come un vero e proprio **linguaggio di modellazione**.

CLASS DIAGRAM

Il **Class Diagram** è uno dei tipi di diagrammi che possono essere inclusi in un modello UML. Si tratta di una rappresentazione logica a grafo, in cui i nodi corrispondono a **classi** e **interfacce**, mentre gli archi rappresentano le **relazioni**.

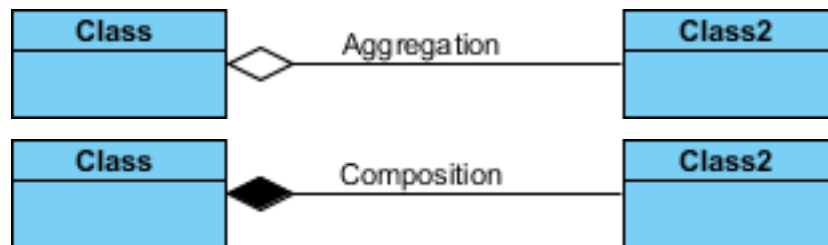


Questa immagine rappresenta un singolo nodo del class diagram - una singola classe.

Esistono diversi tipi di **relazione**, ciascuno dei quali definisce il modo in cui due classi possono interagire all'interno di un modello UML.

I principali tipi di relazione sono:

- **Aggregazione**
- **Associazione**
- **Composizione**
- **Dipendenza**
- **Generalizzazione**
- **Realizzazione**

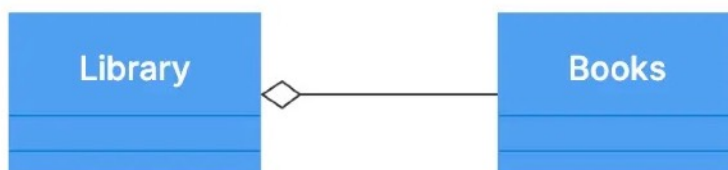


Aggregazione e Composizione sono sottoinsiemi dell'**associazione**, in quanto costituiscono casi specifici di essa. In entrambe, l'oggetto di una classe "possiede" l'oggetto di un'altra classe. Tuttavia, esiste una sottile differenza:

- **Aggregazione:** implica una relazione in cui il figlio può esistere indipendentemente dal genitore.
Esempio: Classe (genitore) e Studente (figlio). Se la Classe aggregante viene eliminata, gli Studenti (classe aggregata) continuano a esistere.
 Questa relazione è definita con il nome di **"Whole-Part"**.
 Il ciclo di vita delle parti non è strettamente accoppiato al ciclo di vita dell'aggregato. Se l'aggregato viene distrutto, le sue parti possono continuare ad esistere. Gli oggetti contenuti (**Part**) possono far parte di più aggregati (**Whole**).
- **Composizione:** implica una relazione in cui il figlio non può esistere indipendentemente dal genitore: ciò significa che le **Part** vengono create quando il **Whole** viene creato e distrutte quando il **Whole** viene distrutto, il che implica dire che questa relazione impone dei vincoli sul **ciclo di vita degli oggetti**.
Esempio: Casa (genitore) e Stanza (figlio). Le Stanze non esistono separatamente da una Casa.
 Ogni **Part** è contenuta in un solo insieme alla volta e non può essere condivisa tra più **Whole**.

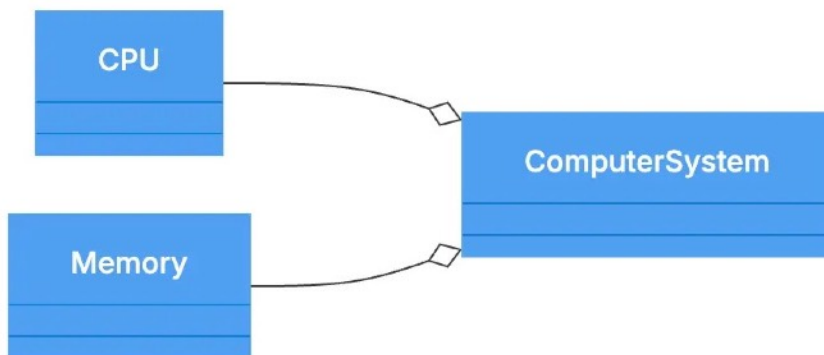
In UML, viene identificata attraverso una **linea continua** con un rombo **bianco** (o **colorato di nero**) dalla parte della classe contenitore (**classe aggregante - Whole**).

Esempio di Aggregazione.



Una biblioteca può avere un rapporto di aggregazione con i libri. La biblioteca (**whole**) contiene libri (**part**), ma i libri non dipendono dalla biblioteca per la loro esistenza. Possono essere trovati in altre biblioteche o posseduti individualmente

Esempio di Aggregazione 2.



In un sistema informatico, un computer (whole) può avere componenti come una CPU e una memoria (parti). Queste parti sono parte integrante del funzionamento del computer ma possono esistere indipendentemente al di fuori del sistema informatico.

Esempio di Composizione.



Un essere umano può essere considerato un tutto, con il cuore come parte. Il cuore non esiste indipendentemente dal corpo umano e il suo ciclo di vita è intrinsecamente legato a quello umano, illustrando una relazione di composizione.

Esempio di Composizione 2.



Una casa (**Whole**) è composto da diverse stanze (**Part**). Le stanze sono parte integrante della struttura della casa e non esistono separatamente da essa. Quando la casa viene demolita, anche le stanze cessano di esistere.

In conclusione, **Composizione** rappresenta un **dipendenza più forte e proprietà** da tutte le sue parti, mentre **Aggregazione** permette di più **flessibilità e indipendenza** di parti.

ESEMPI DI CODICE TRA AGGREGAZIONE E COMPOSIZIONE.

Aggregazione.

```
class Student {

    private String name;

    public Student(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}

class SchoolClass {
    private String subject;
    private List<Student> students; // Aggregazione!!!!!! Occhio

    public SchoolClass(String subject, List<Student> students) {
        this.subject = subject;
        this.students = students;
    }

    public void showStudents() {
        System.out.println("Classe di " + subject + ":");
        for (Student s : students) {
            System.out.println("- " + s.getName());
        }
    }
}
```

```

    }
}

public class AggregationExample {
    public static void main(String[] args) {
        // Gli studenti esistono indipendentemente dalla classe
        Student s1 = new Student("Marco");
        Student s2 = new Student("Giulia");

        List<Student> students = new ArrayList<>();
        students.add(s1);
        students.add(s2);

        SchoolClass mathClass = new SchoolClass("Matematica", students);
        mathClass.showStudents();
    }
}

```

Qui gli **Studenti** esistono anche senza la **SchoolClass**. Se elimini la classe, gli studenti rimangono oggetti validi.

Composizione.

```

class Room {
    private String name;

    public Room(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}

class House {
    private List<Room> rooms; // Composizione!!!! Occhio

    public House() {
        rooms = new ArrayList<>();
        // Le stanze nascono e muoiono con la casa
        rooms.add(new Room("Soggiorno"));
        rooms.add(new Room("Camera da letto"));
        rooms.add(new Room("Cucina"));
    }

    public void showRooms() {
        System.out.println("La casa contiene le seguenti stanze:");
        for (Room r : rooms) {
            System.out.println("- " + r.getName());
        }
    }
}

public class CompositionExample {
    public static void main(String[] args) {

```

```

    House house = new House();
    house.showRooms();
    // Le stanze non possono esistere senza la casa
}

```

Qui le **Stanze** vengono create **dentro la House** e non hanno senso al di fuori di essa. Se la House viene distrutta, anche le stanze spariscono.

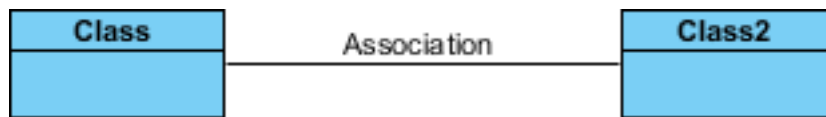
Prima di continuare formalizziamo delle definizioni noiose ma che sono propedeutiche alla lettura.

DEFINIZIONI.

OPERAZIONE VS METODO VS MESSAGGIO

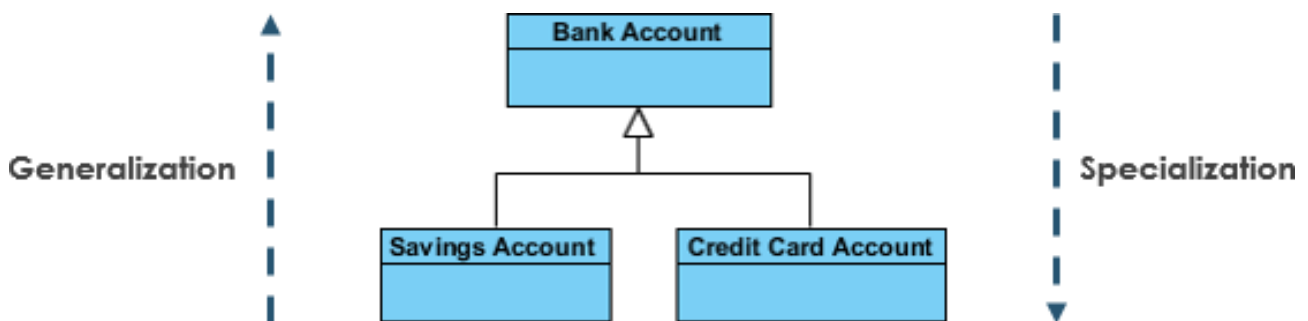
- **Operazione**: l'operazione specifica che “cosa fa” una classe senza specificare il “come”. Le operazioni **manipolano lo stato degli oggetti**, ovvero il *valore degli attributi dell'oggetto*. Ogni operazione ha una **segnatura**, che comprende un nome e una lista di parametri., [NO IL TIPO DI RITORNO] **In sostanza**, specifica di un “*segnatura*” (**prototipo**) che un oggetto mette a disposizione di altri oggetti.
- **Metodo**: è l'implementazione vera e propria di un'operazione messa a disposizione da una classe. Oltre alla segnatura, è caratterizzato anche dal corpo che contiene una o più sequenze di istruzioni scritte per eseguire una determinata azione. **In sostanza**, implementazione vera e propria dell'operazione di un oggetto.
- **Messaggio**: è la richiesta di invocazione a run-time da parte di un oggetto **A** su un metodo fornito da un oggetto **B**.

Continuiamo la lettura.



Se due classi di un modello devono comunicare tra loro, deve esserci un collegamento tra loro e ciò può essere rappresentato da un'associazione. Le istanze di una classe sono collegate a istanze di un'altra classe. Una classe può inviare messaggi a quella associata.

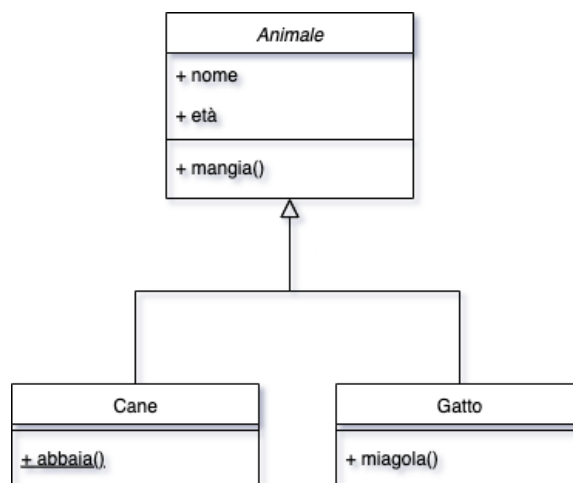
In UML, viene rappresentata con una freccia continua con la punta rivolta verso la classe che può ricevere messaggi (**asimmetrica**). Oppure con una linea se è navigabile nelle due direzioni (**simmetrica**).



Adesso affrontiamo il caso della **relazione di generalizzazione**.

Quando implementiamo la Generalizzazione in un linguaggio di programmazione, spesso viene invece chiamata Eredità. Generalizzazione ed eredità sono le stesse. La terminologia differisce solo a seconda del contesto in cui viene utilizzata.

In UML, la generalizzazione è rappresentata da una linea con una freccia vuota che punta verso la superclasse. <<*i figli estendono la classe padre*>>.



In Java, la relazione di generalizzazione viene implementata usando l'ereditarietà, tramite la parola "EXTENDS".

Prima di vedere il codice, chiariamolo: una sottoclasse eredita **tutte** (e sottolinea tutte) le caratteristiche della superclasse.

```
class Animale {  
  
    public String nome;  
    public int eta;  
  
    public Animale(String nome, int eta) {  
        this.nome = nome;  
        this.eta = eta;  
    }  
  
    public void mangia() {  
        System.out.println(nome + " sta mangiando.");  
    }  
}  
  
class Cane extends Animale {  
  
    public Cane(String nome, int eta) {  
        super(nome, eta);  
    }  
  
    public void abbaia() {  
        System.out.println(nome + " sta abbaiando.");  
    }  
}  
  
class Gatto extends Animale {  
  
    public Gatto(String nome, int eta) {  
        super(nome, eta); // richiama il costruttore della  
superclasse  
    }  
  
    public void miagola() {  
        System.out.println(nome + " sta miagolando.");  
    }  
}
```

```

public class Main {
    public static void main(String[] args) {
        Cane cane = new Cane("Fido", 3);
        Gatto gatto = new Gatto("Micia", 2);

        // Cane
        cane.mangia();
        cane.abbaia();

        // Gatto
        gatto.mangia();
        gatto.miagola();
    }
}

```

Output:

```

Fido sta mangiando.
Fido sta abbaiano.
Micia sta mangiando.
Micia sta miagolando.

```

Cane <<is a kind of> Animale.
 Gatto <<is a kind of> Animale.

Attenzione: Java non supporta l'ereditarietà multipla, una classe non può ereditare da due o più superclassi. In C++ sì, in UML sì.

Principio di Sostituibilità di Liskov

Se S è una sottoclasse di T , allora gli oggetti di tipo T possono essere sostituiti con oggetti di tipo S senza alterare le proprietà desiderabili del programma.

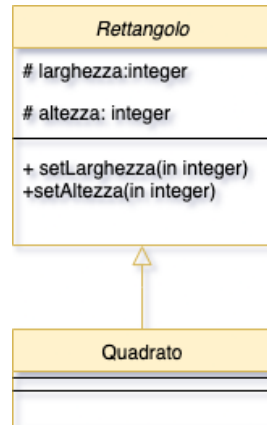
In maniera più dettagliata, se $Q(X)$ è una proprietà che si può dimostrare essere valida per oggetti X di tipo T , allora $Q(Y)$ deve essere valida per oggetti Y di tipo S , dove S è un sottotipo di T .

Nell'esempio appena visto, **Cane** e **Gatto** rispettano il LSP perché possono essere usati al posto di **Animale** senza rompere la logica.

In questo caso posso utilizzare il **Gatto** tutte le volte che ho bisogno di **Animale**.

Se non vale LSP, la frase diventa: non posso utilizzare il **nome_classe_figlia** tutte le volte che ho bisogno di **nome_classe_padre**.

Nota: Qualsiasi **modello** che presenti una relazione di generalizzazione in contrasto con il principio di sostituibilità di Liskov è **errato**.

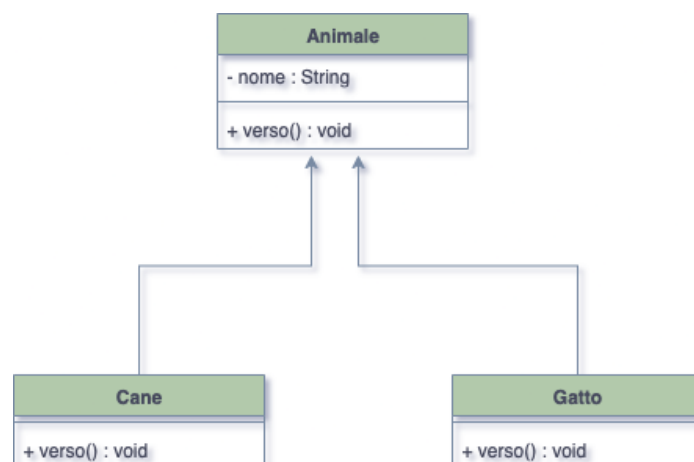


Questo non rispetta LSP.

Qui il Quadrato estende Rettangolo. Non posso utilizzare il Quadrato tutte le volte che ho bisogno di un Rettangolo.

Uno dei motivi principali delle sottoclassi è quello di ridefinire operazioni che hanno ereditato, in gergo “sovrascrivere un metodo”: @override.

Overriding in Java si verifica quando un **sottoclasse implementa un metodo che è già definito nella superclasse**. Il metodo della sottoclasse deve corrispondere a quello della classe madre, ossia devono essere uguali **nome, parametri e tipo di ritorno**.



```

class Animale {
    protected String nome;

    public Animale(String nome) {
        this.nome = nome;
    }

    public void verso() {
        System.out.println(nome + " emette un verso
generico.");
    }
}

// Sottoclasse Cane
class Cane extends Animale {
    public Cane(String nome) {
        super(nome);
    }

    // Override del metodo verso()
    @Override
    public void verso() {
        System.out.println(nome + " sta abbaiando: Bau
Bau!");
    }
}

// Sottoclasse Gatto
class Gatto extends Animale {
    public Gatto(String nome) {
        super(nome);
    }

    // Override del metodo verso()
    @Override
    public void verso() {
        System.out.println(nome + " sta miagolando: Miao!");
    }
}

public class OverrideExample {
    public static void main(String[] args) {
        Animale a1 = new Cane("Fido");
        Animale a2 = new Gatto("Micia");
    }
}

```

```
        // Polimorfismo: chiama i metodi override
        a1.verso();
        a2.verso();
    }
}
```

Output:

Fido sta abbaiano: Bau Bau!

Micia sta miagolando: Miao!

Siamo in presenza di una situazione di polimorfismo.

POLIMORFISMO

È la capacità che ha una classe di comportarsi in modo differente in base al contesto di esecuzione a run-time.

Classe_Padre istanza = new **Classe_Figlia**(arg del costruttore).

(Considero associato il costruttore in questa trattazione).

In questo modo, istanza, può chiamare i metodi specifici sovrascritti nella classe figlia presso cui è stata istanziata. Nel nostro caso il verso, ma il verso specifico di cane o gatto, non generico della superclasse.

In un linguaggio fortemente tipizzato come **Java**, quando si utilizza una variabile di riferimento di tipo **padre** per riferirsi a un'istanza di una classe **figlia**, si è vincolati a invocare esclusivamente i metodi dichiarati nella classe padre. Tuttavia, qualora uno di tali metodi sia stato **sovrascritto** (**override**) nella sottoclasse, l'invocazione effettuata tramite il riferimento di tipo padre determinerà comunque l'esecuzione dell'implementazione fornita dalla classe figlia. Ciò è reso possibile dal meccanismo di **binding dinamico** adottato da Java, il quale consente di selezionare, a tempo di esecuzione, il metodo più specifico in funzione del tipo effettivo dell'istanza puntata dalla variabile, ad esempio **Classe_Figlia**, e non quello della classe di riferimento (**Classe_Padre**).

Si noti che la parola “**Binding**” indica il meccanismo di associazione delle operazioni agli effettivi metodi da eseguire.

La parola “dinamico” indica il fatto che ciò avviene solo a *Run-Time*.

Questo avviene per i metodi **PUBLIC** e **PROTECTED** che possono essere sovrascritti nelle classi figlie.

Fanno eccezione i metodi dichiarati come **PRIVATE**, **STATIC** E **FINAL**.

```
class Animale {
    public void verso() {                // Metodo
sovrascrivibile
        System.out.println("Animale: verso generico");
    }

    private void dorme() {                // Metodo private →
non ereditato
        System.out.println("Animale: sta dormendo");
    }

    public static void respira() {        // Metodo static →
legato alla classe
        System.out.println("Animale: respirazione base");
    }

    public final void muove() {           // Metodo final →
non sovrascrivibile
        System.out.println("Animale: movimento generico");
    }
}

class Cane extends Animale {
    @Override
    public void verso() {                // Override →
binding dinamico
        System.out.println("Cane: Bau Bau!");
    }

    // Questo non è override: è un nuovo metodo,
    // perché dorme() del padre è private e quindi invisibile
    public void dorme() {
        System.out.println("Cane: sta russando");
    }
}
```

```

    // Questo nasconde (hiding) il metodo static della
superclasse
    public static void respira() {
        System.out.println("Cane: respirazione affannata");
    }

    // Se provassi a fare override di muove(), avrei un
errore di compilazione
    /*
    @Override
    public void muove() {
        System.out.println("Cane si muove correndo"); //
Errore: metodo final
    }
    */
}

public class BindingEsempio {
    public static void main(String[] args) {
        Animale a = new Cane(); // Riferimento di tipo padre

        // Binding dinamico → viene invocato il metodo della
sottoclasse
        a.verso();    // Output: Cane: Bau Bau!

        // Muove è final → rimane quello della superclasse
        a.muove();    // Output: Animale: movimento generico

        // Static: dipende dal tipo di riferimento, non
dall'istanza
        a.respira(); // Output: Animale: respirazione base
        Cane.respira(); // Output: Cane: respirazione
affannata

        // Non posso chiamare a.dorme() perché in Animale è
private
        // Se chiamo su un Cane reale:
        Cane c = new Cane();
        c.dorme();    // Output: Cane: sta russando
    }
}

```

I metodi dichiarati come **final** sono progettati per non essere sovrascritti.

Nota che `final` può essere associato anche ad una classe. Public **final** class nome classe, significa che la classe **non può essere estesa**: *impedisce a qualsiasi sottoclasse di ereditare da essa.*

Regola importante: in Java è consentito aumentare la visibilità di un metodo sovrascritto nel figlio rispetto allo stesso metodo della classe padre. Non è consentito ridurre la visibilità, un metodo `protected` nella classe padre può diventare `public` nella classe figlia, ma un metodo `public` nella classe padre non può diventare `protected` o `private` nella classe figlia.

OVERRIDE VS OVERLOADING

Overloading: Si verifica quando abbiamo più metodi nella stessa classe con lo stesso nome ma abbiamo un numero diverso di parametri. Permette di eseguire operazioni con ingressi diversi.

Override (Già VISTO) : Si verifica quando una sottoclasse fornisce un'implementazione specifica per un metodo già definito nella superclasse.

Esempio di Overloading.

```
class Calcolatrice {  
  
    // Metodo 1: somma di due interi  
    public int somma(int a, int b) {  
        return a + b;  
    }  
  
    // Metodo 2: somma di tre interi (overloading sul numero di  
    parametri)  
    public int somma(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    // Metodo 3: somma di due double (overloading sul tipo dei parametri)  
    public double somma(double a, double b) {  
        return a + b;  
    }  
}  
  
public class OverloadingExample {  
    public static void main(String[] args) {  
        Calcolatrice calc = new Calcolatrice();  
  
        System.out.println(calc.somma(2, 3));           // usa metodo 1 → 5  
        System.out.println(calc.somma(2, 3, 4));       // usa metodo 2 → 9  
        System.out.println(calc.somma(2.5, 3.5));     // usa metodo 3 → 6.0  
    }  
}
```


OVERLOADING DEI COSTRUTTORI

In Java, puoi inserire un numero illimitato di costruttori all'interno di una classe, a condizione che ogni costruttore abbia una firma unica. La firma dei costruttori è univocamente determinata dal numero e dal tipo dei parametri.

```
class Persona {
    private String nome;
    private int eta;

    // Costruttore 1: solo nome
    public Persona(String nome) {
        this.nome = nome;
        this.eta = 0; // valore di default
    }

    // Costruttore 2: nome ed età
    public Persona(String nome, int eta) {
        this.nome = nome;
        this.eta = eta;
    }

    // Metodo che stampa qualcosa ar Fly
    public void mostraInfo() {
        System.out.println("Nome: " + nome + ", Età: " + eta);
    }
}

public class OverloadingCostruttore {
    public static void main(String[] args) {
        Persona p1 = new Persona("Marco");           // usa costruttore 1
        Persona p2 = new Persona("Giulia", 25);       // usa costruttore 2

        p1.mostraInfo(); // Output: Nome: Marco, Età: 0
        p2.mostraInfo(); // Output: Nome: Giulia, Età: 25
    }
}
```

INCAPSULAMENTO

In Java, l'incapsulamento è uno dei concetti fondamentali della programmazione orientata agli oggetti (OOP).



Questa immagine qui sopra dimostra il concetto di incapsulamento, dove una classe (rappresentata dalla capsula) nasconde i suoi dati interni (variabili) ed espone solo un'interfaccia controllata (incapsulamento).

Nel Java, l'incapsulamento viene implementato dichiarando le variabili di istanza come **private**, limitando l'accesso diretto. I metodi getter pubblici recuperano valori variabili, mentre i metodi setter li modificano, consentendo l'accesso controllato.

Nell'incapsulamento, le variabili o i dati di una classe sono nascosti da qualsiasi altra classe ed è possibile accedervi solo tramite qualsiasi funzione membro della propria classe.

```
class Person {

    // Encapsulating the name and age
    // only approachable and used using methods defined
    private String name;
    private int age;

    public String getName() { return name; }

    public void setName(String name) { this.name = name; }

    public int getAge() { return age; }

    public void setAge(int age) { this.age = age; }
}

// Driver Class
public class Geeks {

    // main function
    public static void main(String[] args)
    {
        // person object created
        Person p = new Person();
        p.setName("Sweta");
        p.setAge(25);

        // Using methods to get the values from the
        // variables
        System.out.println("Name: " + p.getName());
        System.out.println("Age: " + p.getAge());
    }
}
```

Out:

Name: Sweta

Age: 25

Un altro esempio:

```
class Area {
    private int l; // this value stores length
    private int b; // this value stores breadth

    // constructor to initialize values
    Area(int l, int b)
    {
        this.l = l;
        this.b = b;
    }

    // method to calculate area
    public void getArea()
    {
        int area = l * b;
        System.out.println("Area: " + area);
    }
}

public class Geeks {
    public static void main(String[] args)
    {
        Area rect = new Area(2, 16);
        rect.getArea();
    }
}
```

Out:

Area: 32

INFORMATION HIDING

Sto per introdurre un argomento di particolare rilevanza. Se stai scorrendo questo documento con la leggerezza con cui si sfoglia un giornale, è giunto il momento di fermarti e dedicare la dovuta attenzione alla lettura, al fine di comprenderne appieno i contenuti.

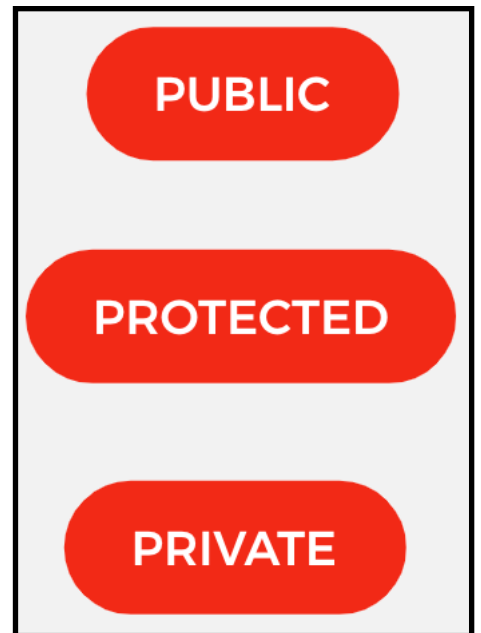
Attenzione.

I termini **incapsulamento** e **information hiding** vengono spesso utilizzati come sinonimi, anche se tra loro **esiste una sottile differenza** concettuale: l'information hiding è il principio teorico su cui si basa la tecnica dell'incapsulamento.

Secondo il concetto di information hiding, i dettagli implementativi di una classe sono nascosti all'utente.

Occorre introdurre il concetto di visibilità.

VISIBILITÀ



Ogni elemento di una classe, che sia un attributo o un metodo, ha una visibilità.

- **Protected** : sono accessibili all'interno della classe e dalle sottoclassi (anche se si trovano in pacchetti diversi) e da tutte le classi nello stesso pacchetto.
- **Private** : sono accessibili solo all'interno della classe in cui sono definiti.
- **Public** : sono accessibili da qualsiasi classe in qualunque pacchetto.

Nota: se non esiste la generalizzazione, **Private=Protected** (sono la stessa cosa).

Esempio: Nel nostro caso, la classe **Main** è nello stesso package della classe **Persona**.

```
class Persona {  
    protected String nome;    // visibile a sottoclassi e stesso package  
    protected int eta;  
  
    public Persona(String nome, int eta) {  
        this.nome = nome;  
        this.eta = eta;  
    }  
  
    protected void saluta() {  
        System.out.println("Ciao, mi chiamo " + nome + " e ho " + eta + "  
anni.");  
    }  
}
```

```

public class Main {
    public static void main(String[] args) {
        Persona p = new Persona("Marco", 25);

        // Nonostante siano protected, qui posso accedere
        // perché Main si trova nello stesso package di Persona
        System.out.println("Nome: " + p.nome);
        System.out.println("Età: " + p.eta);

        p.saluta();
    }
}

```

Esempio: Tutti package diversi.

```

package studenti;

import persone.Persona;

public class Studente extends Persona {

    private String corso;

    public Studente(String nome, int eta, String corso) {
        super(nome, eta);
        this.corso = corso;
    }

    public void mostraInfo() {
        // Posso accedere a nome e eta perché sono protected
        System.out.println("Studente: " + nome + ", " + eta + " anni, Corso: " +
corso);

        // Posso richiamare anche il metodo protected
        saluta();
    }
}

```

```

package main;

import studenti.Studente;

public class Main {
    public static void main(String[] args) {
        Studente s = new Studente("Giulia", 21, "Informatica");

        s.mostraInfo();

        // ❌ Qui NON posso accedere a s.nome o s.eta direttamente
        // perché Main è in un package diverso e non è una sottoclasse
        // System.out.println(s.nome); // Errore di compilazione
    }
}

```

```

package persone;

```

```

public class Persona {
    protected String nome;
    protected int eta;

    public Persona(String nome, int eta) {
        this.nome = nome;
        this.eta = eta;
    }

    protected void saluta() {
        System.out.println("Ciao, mi chiamo " + nome + " e ho " + eta + "
anni.");
    }
}

```

Visibilità in UML

- **-** (meno) → private .
- **+** (più) → public .
- **#** (cancellotto) → protected .

Esempio.

```

+-----+
|          Persona          |
+-----+
| # nome : String          |
| # eta : int              |
+-----+
| + Persona(nome, eta)     |
| # saluta() : void        |
+-----+

```

nome ed **eta** sono protected (#).

Il costruttore è public (+).

Il metodo `saluta()` è protected (#).

CLASSI ASTRATTE

Immagina di essere **Dio**.

All'inizio dell'universo esisti solo tu.

Decidi allora di creare altri pianeti, ma non vuoi che siano *tutti uguali*: vuoi stabilire delle **regole generali** che ogni pianeta dovrà rispettare, pur lasciando libertà di personalizzazione.



Dio: “Da oggi esisterà il concetto di **Pianeta**. Ogni pianeta deve avere un **nome**, una **dimensione** e deve poter **ruotare attorno a una stella**”.

Ma attenzione:

- **Pianeta** è solo un'**idea** (una classe astratta): non puoi “abitare” direttamente su un concetto!
- Puoi però creare pianeti specifici (classi concrete) che *ereditano* dal concetto di Pianeta, come **Terra**, **Marte**, **Giove**, ecc.
- Ognuno di questi pianeti concretizza i dettagli: atmosfera, gravità, vita, ecc.

Ecco spiegato che cos'è una **classe astratta**.

Analogia riassuntiva:

Dio = il programmatore.

Pianeta = **classe astratta** → un concetto generale che non esiste da solo.

Terra, Marte, ecc. = **classi concrete** → realizzazioni specifiche che rispettano le regole generali ma con dettagli propri.

Teorema: in Java (e in tutti i linguaggi OOP classici) **non puoi fare `new` di una classe astratta.**

Le classi astratte in Java sono dichiarate con la parola chiave “**abstract**”.

Una classe astratta può avere al suo interno **metodi astratti**, ovvero **metodi che sono dichiarati ma che non hanno un corpo**. Questi metodi devono essere implementati nelle classi figlie.

Questo obbliga le classi figlie a fornire una versione specifica di quel metodo.

Una classe astratta può anche avere metodi concreti che possono essere utilizzati direttamente dalle classi figlie.

Una classe astratta può non avere alcun metodo, nemmeno metodi astratti.

```
abstract class Pianeta {  
    // Nessun attributo, nessun metodo  
}
```

Esempio sugli animali:

```
abstract class Animale {  
    // Metodo astratto → senza corpo  
    public abstract void verso();  
  
    // Metodo concreto → con corpo  
    public void dorme() {  
        System.out.println("L'animale sta dormendo.");  
    }  
}
```

```
class Cane extends Animale {  
    @Override  
    public void verso() {  
        System.out.println("Bau Bau!");  
    }  
}
```



```

class Gatto extends Animale {
    @Override
    public void verso() {
        System.out.println("Miao!");
    }
}

public class Main {
    public static void main(String[] args) {
        Animale a1 = new Cane();
        Animale a2 = new Gatto();

        a1.verso(); // Output: Bau Bau!
        a2.verso(); // Output: Miao!
        a1.dorme(); // Output: L'animale sta dormendo.
    }
}

```

Esempio di classe astratta vuota:

Classe astratta vuota → funziona solo come “contenitore comune”, ma per chiamare metodi serve il **casting**.

```

// Classe astratta vuota: rappresenta un concetto generico di Animale
abstract class Animale {
    // Nessun attributo, nessun metodo
}

// Classe concreta Cane
class Cane extends Animale {
    public void verso() {
        System.out.println("Il cane abbaia: Bau Bau!");
    }
}

// Classe concreta Gatto
class Gatto extends Animale {
    public void verso() {
        System.out.println("Il gatto miagola: Miao!");
    }
}

public class Main {

```

```

    public static void main(String[] args) {
        // Polimorfismo: stesso tipo di riferimento, istanze
diverse
        Animale a1 = new Cane();
        Animale a2 = new Gatto();

        // Casting per accedere ai metodi specifici
        ((Cane) a1).verso();    // Output: Il cane abbaia: Bau
Bau!
        ((Gatto) a2).verso();  // Output: Il gatto miagola:
Miao!
    }
}

```

Se la **classe astratta vuota** non definisce alcun metodo, per usare il **polimorfismo senza casting** dobbiamo darle almeno un **metodo astratto** comune (ad esempio verso()).

```

// Classe astratta con un metodo astratto
abstract class Animale {
    public abstract void verso(); // obbliga le sottoclassi a
implementarlo
}

// Classe concreta Cane
class Cane extends Animale {
    @Override
    public void verso() {
        System.out.println("Il cane abbaia: Bau Bau!");
    }
}

// Classe concreta Gatto
class Gatto extends Animale {
    @Override
    public void verso() {
        System.out.println("Il gatto miagola: Miao!");
    }
}

public class Main {
    public static void main(String[] args) {
        // Polimorfismo: stesso riferimento, istanze diverse
        Animale a1 = new Cane();
        Animale a2 = new Gatto();
    }
}

```

```

    // Non serve il casting
    a1.verso(); // Output: Il cane abbaia: Bau Bau!
    a2.verso(); // Output: Il gatto miagola: Miao!
  }
}

```

In UML, una pratica comune è aggiungere l'annotazione {abstract} accanto al nome della classe.

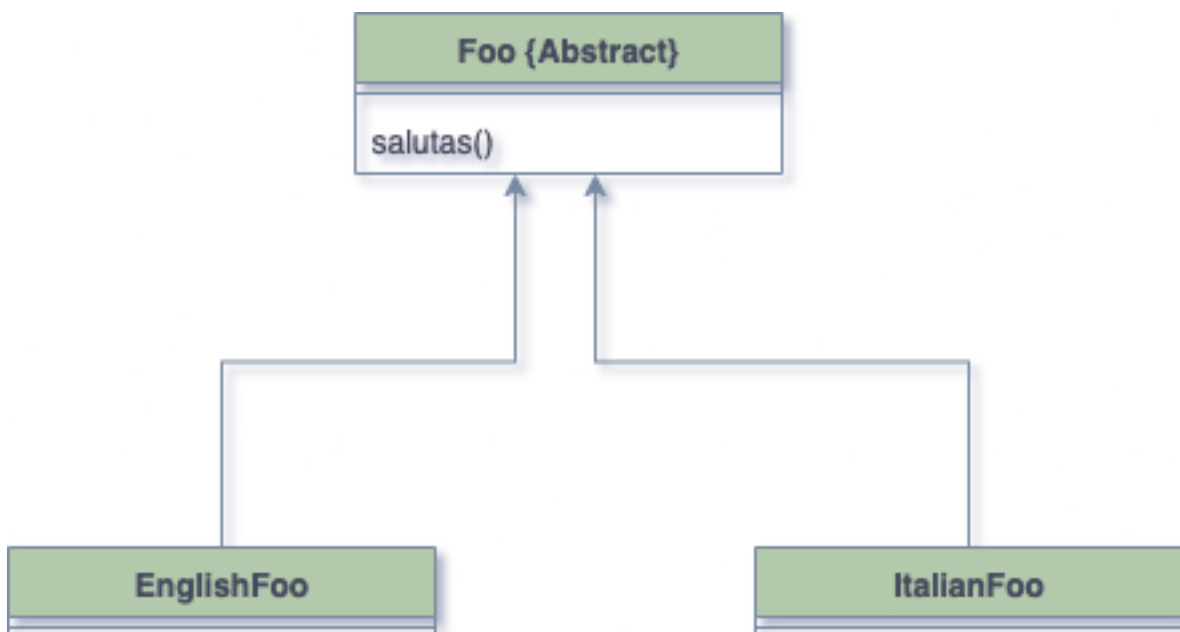
Ed è qui che centriamo il focus più importante rispetto a quello che abbiamo detto.

Per avere una soluzione polimorfa devono essere garantite le seguenti condizioni:

- Gerarchia di Classi;
- Variabile con tipo la superclasse astratta;
- **Meccanismo di scelta** da concretizzarsi solo a Run-Time;
- Linguaggio con supporto di Binding Dinamico.

Nota: non è obbligatorio dichiarare una classe come <<abstract>> per utilizzare il polimorfismo.

Di seguito, il codice del Prof. Guglielmo De Angelis.



```
    Foo f;
    if(<test>) {
        f = new EnglishFoo();
    } else {
        f = new ItalianFoo();
    }
    f.salutas();
```

```
abstract class Foo {
    // Metodo astratto
    public abstract void salutas();
}
```

```
class EnglishFoo extends Foo {
    @Override
    public void salutas() {
        System.out.println("Hello!");
    }
}
```

```
class ItalianFoo extends Foo {
    @Override
    public void salutas() {
        System.out.println("Ciao!");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Foo f;

        boolean test = true; // cambia questo per provare

        if (test) {
            f = new EnglishFoo();
        } else {
            f = new ItalianFoo();
        }

        f.salutas(); // Output: "Hello!" se test = true,
        "Ciao!" se test = false
    }
}
```

Quindi, sapendo tutto questo, posso asserire che
abstract + final è una contraddizione logica:
abstract dice: "questa classe deve essere estesa".
final dice: "questa classe non può essere estesa".

CLASSI NIDIFICATE

Una classe nidificata è una classe dichiarata all'interno di un'altra classe.

```
class Esterna {
    private String messaggio = "Ciao dal campo privato!";

    // Classe statica nidificata
    static class Statica {
        public void stampa() {
            System.out.println("Sono una classe statica
nidificata!");
        }
    }

    // Classe interna non statica
    class Interna {
        public void stampa() {
            // Può accedere a campi anche private della
classe esterna
            System.out.println("Messaggio dalla classe
interna: " + messaggio);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        // Uso della classe statica nidificata
        Esterna.Statica s = new Esterna.Statica();
        s.stampa();

        // Uso della classe interna (serve un'istanza della
classe esterna)
        Esterna e = new Esterna();
    }
}
```

```

        Esterna.Interna i = e.new Interna();
        i.stampa();
    }
}

```

Quindi, una classe annidata statica può essere creata senza istanziare l'oggetto.
Invece per usare interna dobbiamo prima creare esterna.

out:

Sono una classe statica nidificata!
Messaggio dalla classe interna: Ciao dal campo privato!

Nota: Una classe interna non statica in Java può accedere direttamente anche agli **attributi private** della classe esterna.

Nota 2; Una nested class statica non ha alcun riferimento all'istanza esterna, quindi può accedere solo ai membri **static** della classe esterna.

```

class Esterna {
    private String messaggio = "Ciao da istanza!";
    private static String messaggioStatic = "Ciao da static!";

    // Inner class non statica
    class Interna {
        public void stampa() {
            // Può accedere a TUTTO, anche ai campi private
            System.out.println("Interna → " + messaggio);
            System.out.println("Interna → " + messaggioStatic);
        }
    }

    // Classe nidificata statica
    static class Statica {
        public void stampa() {
            // ❌ Non posso accedere a messaggio (non static)
            // System.out.println(messaggio); // ERRORE

            // ✅ Posso accedere a membri static
            System.out.println("Statica → " + messaggioStatic);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        // Uso dell'interna
        Esterna e = new Esterna();
        Esterna.Interna i = e.new Interna();
        i.stampa();
    }
}

```

```

        // Uso della statica
        Esterna.Statica s = new Esterna.Statica();
        s.stampa();
    }
}

```

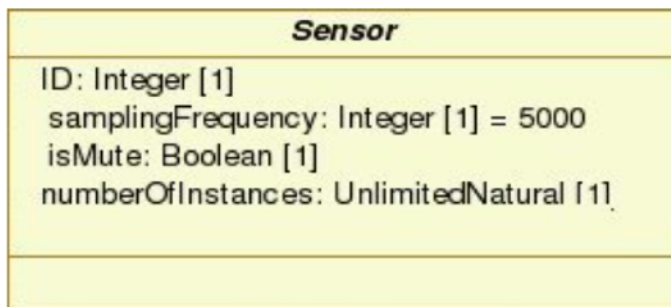
Out:

```

Interna → Ciao da istanza!
Interna → Ciao da static!
Statica → Ciao da static!

```

Una piccola delucidazione in UML:



Quella notazione [1] che vedete accanto agli attributi nella classe Sensor in UML rappresenta la molteplicità dell'attributo, cioè quante istanze di quel valore possono essere associate a un oggetto della classe.

Ogni Sensor deve avere esattamente **1 ID** di tipo Integer.

Ogni Sensor ha esattamente 1 frequenza di campionamento (Integer), con valore di default 5000.

Ogni Sensor ha esattamente **1 attributo booleano** che indica se è muto o meno.

REGOLA:

[1] → esattamente uno (obbligatorio).

[0 .. 1] → opzionale, può mancare o esserci una sola istanza.

[0 .. *] o [*] → zero o molti (lista, collezione).

[1 .. *] → almeno uno.

Ma:

```
+-----+
|  Persona  |
+-----+
| nome : String [1] |
| cognome : String [1] |
| telefoni : String [*] |
+-----+
```

Interpretazione:

- Ogni persona deve avere **esattamente un nome** ([1]).
- Ogni persona deve avere **esattamente un cognome** ([1]).
- Ogni persona può avere **zero, uno o più numeri di telefono** ([*]).

In codice:

```
class Persona {
    private String nome;
    private String cognome;
    private List<String> telefoni; // Molteplicità [*]
```

Ma:

```
+-----+
|  Persona  |
+-----+
| nome : String [1] |
```


| cognome : String [1] |
| codiceFiscale : String[0..1] |
+-----+

Ogni persona ha **esattamente un nome** ([1]).

- Ogni persona ha **esattamente un cognome** ([1]).
- Ogni persona **può avere 0 o 1 codice fiscale** ([0 . . 1]). → cioè è un attributo **opzionale**.

```
class Persona {
    private String nome;
    private String cognome;
    private String codiceFiscale; // [0..1] opzionale

    public Persona(String nome, String cognome) {
        this.nome = nome;
        this.cognome = cognome;
        this.codiceFiscale = null; // opzionale, inizialmente
assente
    }

    public void setCodiceFiscale(String codiceFiscale) {
        this.codiceFiscale = codiceFiscale;
    }

    public void mostraInfo() {
        System.out.println("Nome: " + nome + " " + cognome);
        if (codiceFiscale != null) {
            System.out.println("Codice Fiscale: " +
codiceFiscale);
        } else {
            System.out.println("Codice Fiscale: non
disponibile");
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Persona p1 = new Persona("Giulia", "Verdi");
        Persona p2 = new Persona("Luca", "Bianchi");
        p2.setCodiceFiscale("LCBNCH90A01H501Z");
    }
}
```

```
        p1.mostraInfo();  
        p2.mostraInfo();  
    }  
}
```

PROBLEMA DELLA FRAGILITÀ DELLA CLASSE PADRE.

Quando si apportano modifiche a una classe padre, è necessario assicurarsi che tali cambiamenti non compromettano il corretto funzionamento delle classi figlie.

Un'evoluzione della classe padre può infatti generare problemi nelle sottoclassi.

Pertanto, intervenire su una classe padre richiede particolare cautela, poiché non è possibile prevedere con certezza in che modo le classi figlie ne abbiano fatto uso.

All'interno di un Class Diagram esiste il concetto di **INTERFACCIA**.



Immagina di svegliarti al mattino, stanco e annoiato.
Prendi una penna e inizi a organizzare la giornata su un foglio:

“Oggi farò questo, alle 12 farò quest'altro, cucinerò, mangerò...”

In quel momento **stai definendo un piano**: scrivi **cosa** farai, senza però specificare **come** farai ciascuna attività.

- Hai deciso che cucinerai, ma non hai ancora stabilito **cosa cucinerai**.
- Hai deciso che mangerai, ma non sai ancora **cosa mangerai**.

Questo è esattamente ciò che fa un'**interfaccia** in programmazione:

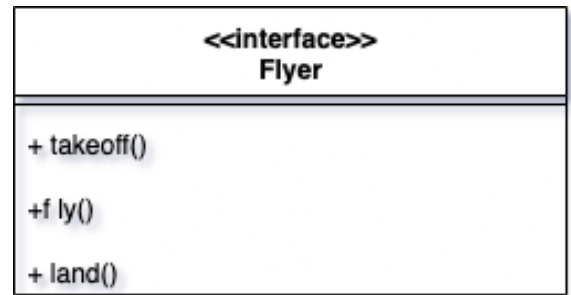
- Definisce **le azioni disponibili** (i metodi).
- Non specifica **come** verranno realizzate, questo lo decideranno le classi che implementano l'interfaccia.

In OOP, un'interfaccia è una struttura che definisce un insieme di metodi senza implementazione. Un'interfaccia stabilisce “cosa” una classe deve essere in grado di fare, ma non dice “come” questi metodi devono essere implementati.

In Java, l'interfaccia è definita con la parola chiave interface.

In UML, l'interfaccia è così rappresentata:

```
public interface Flyer {  
    void takeoff();  
    void fly();  
    void land();  
}
```



Diciamolo subito per evitare equivoci.

I metodi `takeoff()`, `fly()` e `land()` sono **pubblici e astratti per default** nelle interfacce, quindi non serve scrivere `public abstract` (anche se è consentito).

In pratica, chi implementa questa interfaccia (es. *Airplane*, *Bird*, *Superman*) sarà obbligato a definire il comportamento di questi tre metodi.

In **Java 8** è stata introdotta la possibilità di definire **metodi default** all'interno delle interfacce.

Sono metodi **con un corpo** (implementazione) dentro un'interfaccia.

Servono per aggiungere nuove funzionalità alle interfacce **senza rompere** il codice delle classi che già le implementano.

Le classi che implementano l'interfaccia possono usare direttamente il metodo **default**, oppure **sovrascriverlo (override)** se vogliono un comportamento personalizzato.

```
interface Flyer {  
    void takeoff();  
    void fly();  
    void land();  
  
    // Metodo default, ha corpoooo  
    default void checkSystems() {  
        System.out.println("Controllo dei sistemi prima del  
volo...");  
    }  
}
```

Il metodo default è public per default. (Bel gioco di parole, ma utile per ricordare.).

Da java 9 è possibile avere i metodi private nelle interfacce: questi metodi possono avere un'implementazione ma possono essere usati solo all'interno dell'interfaccia stessa.

Esempio 1.

```
interface Flyer {
    void takeoff();
    void fly();
    void land();

    // Metodo default → ha un corpo
    default void checkSystems() {
        System.out.println("Controllo dei sistemi prima del volo...");
    }
}

class Airplane implements Flyer {
    @Override
    public void takeoff() {
        System.out.println("L'aereo decolla dalla pista.");
    }

    @Override
    public void fly() {
        System.out.println("L'aereo sta volando ad alta quota.");
    }

    @Override
    public void land() {
        System.out.println("L'aereo atterra sulla pista.");
    }

    // Override opzionale del metodo default
    @Override
    public void checkSystems() {
        System.out.println("Controllo avanzato dei sistemi dell'aereo.");
    }
}

public class Main {
    public static void main(String[] args) {
        Flyer f = new Airplane();
        f.checkSystems(); // Usa l'override della classe
        f.takeoff();
        f.fly();
        f.land();
    }
}
```

Out:

```
Controllo avanzato dei sistemi dell'aereo.
L'aereo decolla dalla pista.
L'aereo sta volando ad alta quota.
L'aereo atterra sulla pista.
```

Esempio 2.

```
interface Flyer {
    void takeoff();
    void fly();
    void land();

    // Metodo default che usa un metodo private
    default void checkPreFlight() {
        System.out.println("Avvio controllo pre-volo...");
        log("Sistemi controllati.");
    }

    // Metodo statico che usa un private static
    static void maintenance() {
        System.out.println("Avvio manutenzione standard...");
        logStatic("Manutenzione completata.");
    }

    // Metodo privato → usato solo da metodi default
    private void log(String message) {
        System.out.println("[LOG Default] " + message);
    }

    // Metodo privato statico → usato solo da metodi statici
    private static void logStatic(String message) {
        System.out.println("[LOG Static] " + message);
    }
}

class Airplane implements Flyer {
    @Override
    public void takeoff() { System.out.println("Decollo in corso."); }
    @Override
    public void fly() { System.out.println("Sto volando."); }
    @Override
    public void land() { System.out.println("Atterraggio completato."); }
}

public class Main {
    public static void main(String[] args) {
        Flyer f = new Airplane();

        f.takeoff();
        f.fly();
        f.checkPreFlight(); // Usa log() interno
        f.land();

        Flyer.maintenance(); // Usa logStatic() interno
    }
}
```

Out:

Decollo in corso.

Sto volando.
Avvio controllo pre-volo...
[LOG Default] Sistemi controllati.
Atterraggio completato.
Avvio manutenzione standard...
[LOG Static] Manutenzione completata.

Teorema: l'interfaccia può essere implementata da qualsiasi classe.

Non esiste una restrizione che limiti l'implementazione di un'interfaccia solo alle classi “padre” o solo alle classi “figlie”. Qualsiasi classe, indipendentemente dalla sua posizione nella gerarchia, può implementare una o più interfacce.

Una classe può anche implementare **più interfacce contemporaneamente** (ereditarietà multipla di interfacce), senza limitazioni.

Corollario: L'eredità multipla tra interfacce è supportata, l'eredità multipla tra classi (IN JAVA) non è supportata.

Conseguenza: una classe figlia può implementare un'interfaccia anche se la classe padre non la implementa.

ORA FACCIAMO MOLTA ATTENZIONE.

Una classe figlia può estendere una classe padre e allo stesso tempo implementare un'interfaccia.
Se il figlio implementa l'interfaccia, allora è obbligato a definire i metodi dell'interfaccia.

Esempio.

Qui abbiamo generalizzazione + implementazione:

- Animale è la **classe padre** (generalizzazione).
- Uccello la **sottoclasse** che eredita da Animale.
- Volante è un'**interfaccia** che Uccello implementa.

// Interfaccia Volante

```
interface Volante {  
    void decolla();  
    void atterra();  
    void vola();  
    void plana();  
    void faCapriola();  
}
```

```

}

// Classe astratta Animale
abstract class Animale {
    public void dorme() {
        System.out.println("L'animale sta dormendo.");
    }

    public void mangia() {
        System.out.println("L'animale sta mangiando.");
    }
}

// Classe Uccello che estende Animale e implementa Volante
class Uccello extends Animale implements Volante {

    @Override
    public void decolla() {
        System.out.println("L'uccello decolla agitando le ali.");
    }

    @Override
    public void atterra() {
        System.out.println("L'uccello atterra leggero sul ramo.");
    }

    @Override
    public void vola() {
        System.out.println("L'uccello vola alto nel cielo.");
    }

    @Override
    public void plana() {
        System.out.println("L'uccello plana sfruttando il vento.");
    }

    @Override
    public void faCapriola() {
        System.out.println("L'uccello fa una capriola in aria!");
    }
}

// Classe di test
public class Main {
    public static void main(String[] args) {
        Uccello u = new Uccello();

        // Metodi ereditati da Animale
        u.dorme();
        u.mangia();

        // Metodi definiti da Volante
        u.decolla();
        u.vola();
        u.plana();
        u.faCapriola();
    }
}

```

```
        u.atterra();
    }
}
```

Out:

L'animale sta dormendo.

L'animale sta mangiando.

L'uccello decolla agitando le ali.

L'uccello vola alto nel cielo.

L'uccello plana sfruttando il vento.

L'uccello fa una capriola in aria!

L'uccello atterra leggero sul ramo.

ESEMPIO CON IL POLIMORFISMO.

```
// Interfaccia Volante
interface Volante {
    void decolla();
    void vola();
    void atterra();
}

// Classe astratta Animale
abstract class Animale {
    public void dorme() {
        System.out.println("L'animale sta dormendo.");
    }

    public void mangia() {
        System.out.println("L'animale sta mangiando.");
    }
}

// Classe Uccello che estende Animale e implementa Volante
class Uccello extends Animale implements Volante {
    @Override
    public void decolla() {
```



```

        System.out.println("L'uccello decolla.");
    }

    @Override
    public void vola() {
        System.out.println("L'uccello vola.");
    }

    @Override
    public void atterra() {
        System.out.println("L'uccello atterra.");
    }

    public void costruisciNido() {
        System.out.println("L'uccello costruisce un nido.");
    }
}

// Classe Aereo che NON è un animale ma implementa Volante
class Aereo implements Volante {
    @Override
    public void decolla() {
        System.out.println("L'aereo decolla dalla pista.");
    }

    @Override
    public void vola() {
        System.out.println("L'aereo vola tra le nuvole.");
    }

    @Override
    public void atterra() {
        System.out.println("L'aereo atterra sulla pista.");
    }
}

public class Main {
    public static void main(String[] args) {
        // ✅ Polimorfismo con riferimento Volante
        Volante v1 = new Uccello(); //INTERACCIA=CLASSEPADRE
        Volante v2 = new Aereo(); //INTERACCIA=CLASSEPADRE

        v1.decolla(); // OK
        v1.vola(); // OK
    }
}

```

```

v1.atterra(); // OK

v2.decolla(); // OK
v2.vola();    // OK
v2.atterra(); // OK

// ❌ ERRORE: Volante non ha i metodi di Animale
// v1.dorme();
// v1.mangia();

// ✅ Polimorfismo con riferimento Animale
Animale a = new Uccello();
a.dorme(); // OK
a.mangia(); // OK

// ❌ ERRORE: Animale non ha i metodi di Volante
// a.decolla();
// a.vola();
// a.atterra();

// ❌ ERRORE: non posso istanziare un'interfaccia o
una classe astratta
// Volante v = new Volante();
// Animale an = new Animale();

// ❌ ERRORE: il riferimento Volante non vede metodi
specifici di Uccello
// v1.costruisciNido();

// ✅ Serve casting esplicito per accedere a metodi
della sottoclasse
((Uccello) v1).costruisciNido();
}
}

```

Nota: Se la sottoclasse (Uccello) non sovrascrive (override) un metodo della superclasse (Animale), allora viene eseguito il metodo ereditato dalla superclasse. Se invece Uccello fa l'override di `dorme()` o `mangia()`, allora, anche se il riferimento è di tipo Animale, verrà eseguita l'implementazione della sottoclasse.

Esempio.

Se è la classe padre a implementare un'interfaccia, allora tutte le sottoclassi la ereditano automaticamente.

```
interface Volante {
    void decolla();
    void vola();
    void atterra();
}

class Animale implements Volante {
    @Override
    public void decolla() {
        System.out.println("L'animale prova a decollare.");
    }

    @Override
    public void vola() {
        System.out.println("L'animale vola genericamente.");
    }

    @Override
    public void atterra() {
        System.out.println("L'animale atterra.");
    }

    public void mangia() {
        System.out.println("L'animale sta mangiando.");
    }
}

class Pipistrello extends Animale {
    @Override
    public void vola() {
        System.out.println("Il pipistrello vola di notte con l'eco-localizzazione.");
    }

    public void dormeCapovolto() {
        System.out.println("Il pipistrello dorme a testa in giù.");
    }
}

public class Main {
```

```

public static void main(String[] args) {
    // Uso del riferimento di tipo Animale
    Animale a = new Pipistrello();

    a.decolla();    // ereditato da Animale
    a.vola();       // override in Pipistrello
    a.atterra();    // ereditato da Animale
    a.mangia();     // metodo di Animale

    // ❌ Non posso fare:
    // a.dormeCapovolto(); // Errore: metodo non definito
    // in Animale

    // ✅ Serve un cast esplicito per accedere a metodi
    // specifici della sottoclasse
    ((Pipistrello) a).dormeCapovolto();
}
}

```

REALIZZAZIONE

In UML, la realizzazione (*realization*) è la relazione che rappresenta quando una classe concreta fornisce un'implementazione a ciò che è definito in un'interfaccia.

Una classe si impegna a **implementare** le operazioni dichiarate in un'interfaccia (o in una classe astratta). → linea **tratteggiata** con triangolo vuoto.

Uccello -----▷ Volante.

In UML si rappresenta con una linea **tratteggiata** con un **triangolo vuoto**.

In UML, un metodo astratto viene indicato scrivendo il nome del metodo in corsivo.

Esempio ancora più generale.

Nel grafico UML di sotto c'è un bellissimo esempio.

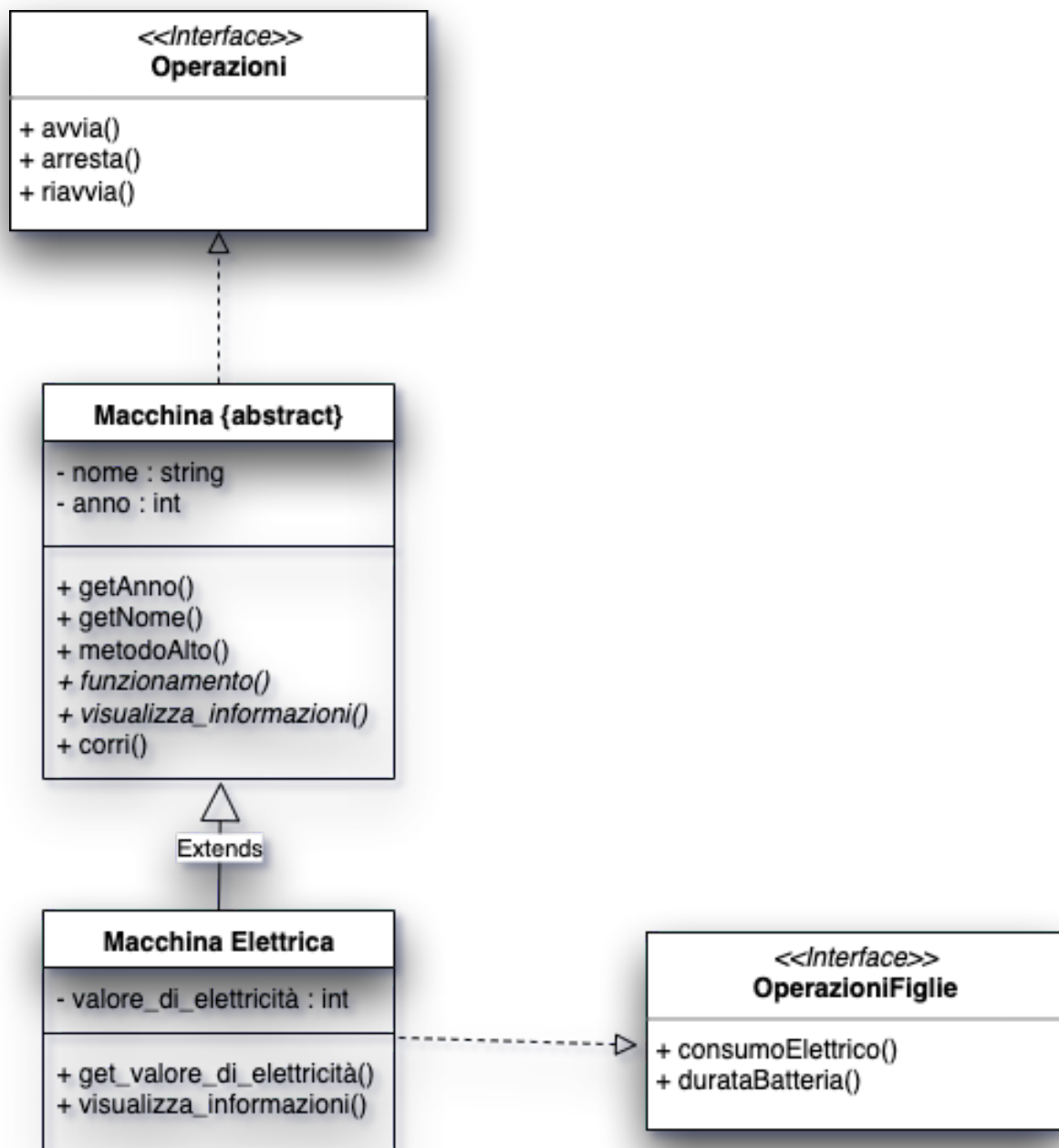
L'interfaccia **Operazioni** definisce i metodi generici che ogni macchina deve saper fare (avvia, arresta, riavvia).

La classe **astratta Macchina** realizza questa interfaccia, quindi ogni sottoclasse erediterà tali operazioni e potrà specializzarle.

La classe **Macchina Elettrica** è una specializzazione di **Macchina** (generalizzazione/ereditarietà).

Oltre ai metodi e attributi ereditati, aggiunge il concetto di `valore_di_elettricità`.

Inoltre, realizza l'interfaccia **OperazioniFiglie**, che specifica funzionalità proprie delle macchine elettriche (`consumoElettrico`, `durataBatteria`).



Se il figlio ridefinisce un metodo del padre, allora col polimorfismo vedi la versione del figlio anche se il riferimento è del padre.

Se il figlio aggiunge un metodo che non esiste nel padre, allora dal riferimento del padre **non puoi chiamarlo**, a meno di fare un **casting**.

```
// Interfaccia con operazioni generiche
interface Operazioni {
    void avvia();
    void arresta();
    void riavvia();
}

// Interfaccia con operazioni specifiche delle macchine elettriche
interface OperazioniFiglie {
    void consumoElettrico();
    void durataBatteria();
}
```

```

}

// Classe astratta Macchina
abstract class Macchina implements Operazioni {
    protected String nome;
    protected int anno;

    public Macchina(String nome, int anno) {
        this.nome = nome;
        this.anno = anno;
    }

    @Override
    public void avvia() {
        System.out.println(nome + " si sta avviando...");
    }

    @Override
    public void arresta() {
        System.out.println(nome + " si è arrestata.");
    }

    @Override
    public void riavvia() {
        System.out.println(nome + " si sta riavviando...");
    }

    // Metodi astratti
    public abstract void visualizzaInformazioni();
    public abstract void funzionamento();
}

// Classe concreta MacchinaElettrica
class MacchinaElettrica extends Macchina implements OperazioniFiglie {
    private int valoreElettricita;

    public MacchinaElettrica(String nome, int anno, int
valoreElettricita) {
        super(nome, anno);
        this.valoreElettricita = valoreElettricita;
    }

    @Override
    public void visualizzaInformazioni() {
        System.out.println("Macchina Elettrica: " + nome + " (" + anno +
"), energia: " + valoreElettricita + " kWh");
    }

    @Override
    public void funzionamento() {
        System.out.println(nome + " funziona grazie alla batteria
elettrica.");
    }

    // Metodi dell'interfaccia OperazioniFiglie
    @Override

```

```

        public void consumoElettrico() {
            System.out.println(nome + " consuma " + (valoreElettricita / 10)
+ " kWh/100km.");
        }

        @Override
        public void durataBatteria() {
            System.out.println(nome + " ha una durata della batteria di " +
valoreElettricita + " km.");
        }
    }

    public class Main {
        public static void main(String[] args) {
            Macchina m = new MacchinaElettrica("Tesla Model S", 2022, 600);

            // ✅ metodi di Operazioni (dichiarati nella superclasse)
            m.avvia();
            m.riavvia();
            m.arresta();

            // ✅ metodi astratti implementati nella sottoclasse
            m.visualizzaInformazioni();
            m.funzionamento();

            // ❌ questi NON funzionano perché Macchina non conosce
OperazioniFiglie
            // m.consumoElettrico();
            // m.durataBatteria();

            // ✅ soluzione 3: CAST a MacchinaElettrica
            ((MacchinaElettrica) m).consumoElettrico();
            ((MacchinaElettrica) m).durataBatteria();

            // ✅ soluzione 4: CAST a OperazioniFiglie
            ((OperazioniFiglie) m).consumoElettrico();
            ((OperazioniFiglie) m).durataBatteria();
        }
    }
}

```

Out:

```

Tesla Model S si sta avviando...
Tesla Model S si sta riavviando...
Tesla Model S si è arrestata.
Macchina Elettrica: Tesla Model S (2022), energia: 600 kWh
Tesla Model S funziona grazie alla batteria elettrica.
Tesla Model S consuma 60 kWh/100km.
Tesla Model S ha una durata della batteria di 600 km.
Tesla Model S consuma 60 kWh/100km.
Tesla Model S ha una durata della batteria di 600 km.

```

Con ((MacchinaElettrica) m) fai un **downcasting**: dici al compilatore “so che in realtà questo oggetto è una MacchinaElettrica”, quindi posso accedere anche ai metodi specifici.

Con ((OperazioniFiglie) m) sfrutti direttamente l'interfaccia implementata dalla sottoclasse.

BUONE PRATICHE

Il downcasting è raro e spesso considerato un code smell.

Nel mondo reale, se un metodo è **comune a tutte le sottoclassi**, si mette nella **superclasse** (o in un'interfaccia comune).

Se è **specifico solo ad alcune sottoclassi**, si lavora direttamente col tipo concreto oppure si definisce un'interfaccia dedicata.

```
OperazioniFiglie me = new MacchinaElettrica("Tesla", 2022, 600);  
me.consumoElettrico();    // non serve casting  
me.durataBatteria();
```

REGOLA PRATICA: SI POSSONO DICHIARARE ANCHE METODI **ABSTRACT** NELLA CLASSE PADRE, EREDITATI DA UN'INTERFACCIA, ED ESPLODERLI NELLE CLASSI FIGLIE.

Le interfacce si sganciano dalla nozione di stato, ci si concentra sulla restrizione delle funzionalità.

Le interfacce non hanno attributi, quindi non c'è una nozione di stato ereditabile, cosa che avviene invece nelle classi astratte.

GERARCHIA DI INTERFACCE

Come le classi, anche le interfacce possono estendere altre interfacce (con **extends**).

Una sott'interfaccia eredita tutti i metodi delle interfacce “padri”.

Una classe concreta che implementa la sott'interfaccia deve implementare tutti i metodi (sia quelli propri che quelli ereditati).

Ora, torniamo al discorso primordiale.

Abbiamo visto la generalizzazione.

RELAZIONE DI DIPENDENZA

È una relazione dove una classe source utilizza o dipende da una classe target.

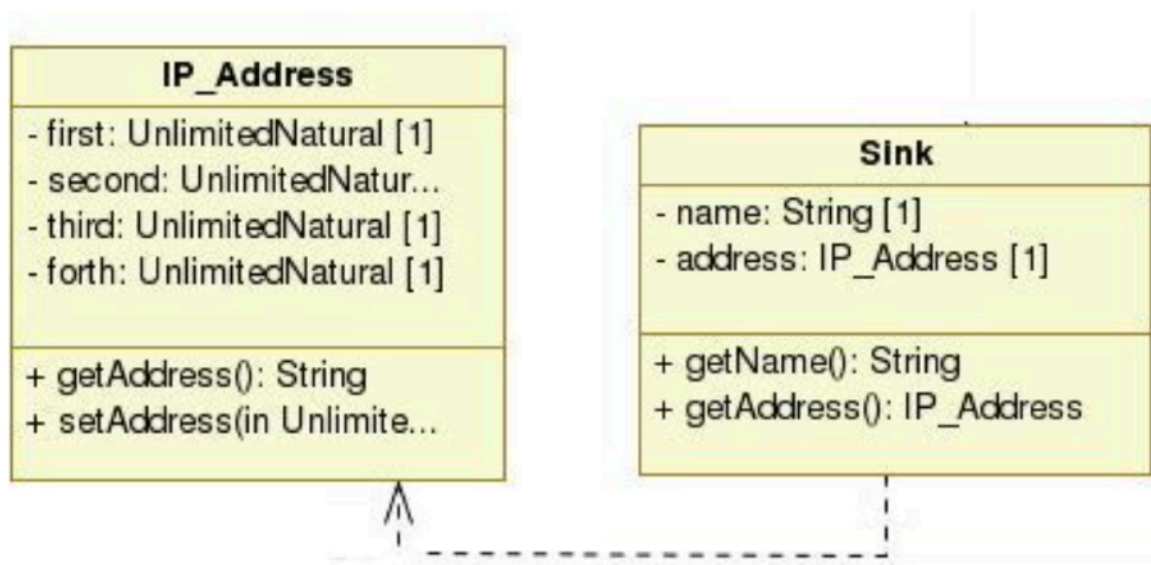
In pratica, sta a indicare che, se la classe target subisce una variazione, allora deve essere cambiata anche l'implementazione della source.

In UML, viene rappresentata mediante una freccia tratteggiata con la punta rivolta verso la classe target.

La dipendenza, all'interno del class diagram, **non è una relazione da esplicitare sempre**, bensì solo quando si vuole avvisare il programmatore del fatto che l'implementazione della classe source dipende fortemente dall'implementazione della classe target, altrimenti il class diagram diverrebbe illeggibile.

Il più delle volte, la relazione di dipendenza indica che la classe source svolge una delle seguenti funzioni:

- Utilizzare temporaneamente la classe target come variabile locale per una delle relative operazioni.
- Inviare un messaggio a un'istanza della classe target.
- Utilizzare temporaneamente la classe target come parametro per una delle relative operazioni.
- Utilizzare temporaneamente la classe target come variabile globale.



Tipo di dipendenza	Stereotipo	Descrizione
Astrazione	<<abstraction>>, <<derive>>, <<refine>> o <<trace>>	Correla due classi che rappresentano lo stesso concetto a differenti livelli di astrazione o da diversi punti di vista.
Sostituzione	<<substitute>>	Indica che la classe source prende il posto del target; la source deve essere conforme al contratto o all'interfaccia stabilita dal target.
Utilizzo	<<use>>, <<call>>, <<create>>, <<instantiate>> o <<send>>	Indica che la classe source richiede la classe target per la relativa implementazione / operatività.

PROGETTAZIONE CON RESPONSABILITÀ.

GRASP.

Nella modellazione orientata agli oggetti (O.O.) si corre il rischio di basarsi solo su abilità personali o su approcci artigianali. Per superare questo limite si utilizza il Responsibility Driven Design (RDD), che assegna a ogni oggetto software delle responsabilità in base al suo ruolo nel sistema.

Le responsabilità possono essere di due tipi:

- **Responsabilità di fare:** creare oggetti, eseguire calcoli, avviare attività di altri oggetti, controllare o coordinare azioni.
- **Responsabilità di conoscere:** mantenere i propri dati incapsulati e conoscere informazioni sugli oggetti collegati o su valori derivabili.

Lo strumento GRASP (General Responsibility Assignment Software Patterns) si basa sui concetti di RDD e fornisce principi utili per supportare la progettazione orientata agli oggetti, migliorando la documentazione e standardizzando i modelli di programmazione.

GRASP comprende nove pattern principali:

1. Creator
2. Information Expert
3. Low Coupling
4. High Cohesion
5. Controller

6. Polymorphism
7. Pure Fabrication
8. Indirection
9. Protected Variations

Non sempre è possibile applicarli tutti contemporaneamente: spetta al progettista scegliere le soluzioni più adatte al contesto.

Un **pattern** è la coppia *problema + soluzione*, codificata e riutilizzabile in diversi contesti. Non introduce concetti nuovi, ma raccoglie e formalizza pratiche già sperimentate e consolidate.

Creator (GRASP Pattern)

Problema:

Chi deve essere responsabile della creazione di una nuova istanza di una classe?

Idea:

La creazione degli oggetti è un'attività comune nei sistemi orientati agli oggetti. È quindi necessario un principio generale che assegni chiaramente la **responsabilità di creazione**.

Vantaggi se applicato correttamente:

- **Basso accoppiamento**
- **Maggiore chiarezza**
- **Incapsulamento**
- **Riusabilità**

Soluzione (Creator)

La classe **B** deve essere responsabile della creazione di un'istanza della classe **A** se almeno una delle seguenti condizioni è vera:

- **B contiene o aggrega** oggetti di tipo **A**.
- **B registra A**.

- **B usa strettamente A.**
- **B possiede i dati necessari** all'inizializzazione di A.

In questi casi, B è detto **creatore di A**.

Se più classi potrebbero creare A, si sceglie quella che:

1. **contiene o aggrega A**,
2. oppure soddisfa il **maggior numero di condizioni** sopra elencate.

Information Expert

Problema:

Come assegnare in modo corretto le responsabilità agli oggetti?

Idea:

In un sistema orientato agli oggetti, ci sono molte classi e molte responsabilità. Per mantenere il sistema semplice, estensibile e riusabile, bisogna assegnare ogni responsabilità alla classe che **ha le informazioni necessarie per soddisfarla**.

Soluzione:

Attribuire la responsabilità all'**esperto delle informazioni**, cioè all'oggetto che già possiede i dati necessari.

Esempio:

Chi deve calcolare il totale di una vendita?

- La classe **Vendita** possiede la lista dei prodotti.
- La classe **Prodotto** conosce il proprio prezzo.
Quindi la responsabilità del calcolo del totale spetta a **Vendita**, che somma i prezzi dei prodotti (ognuno dei quali conosce il proprio valore).

Low Coupling

Problema:

Come ridurre le dipendenze tra classi per diminuire l'impatto dei cambiamenti e aumentare le opportunità di riuso?

Concetto:

L'**accoppiamento** (**coupling**) misura quanto una classe è connessa ad altre classi e da esse dipende.

Un forte accoppiamento è problematico perché:

- Ogni modifica in una classe correlata impone modifiche anche alle altre.
- Le classi diventano più difficili da comprendere in isolamento.
- Il riuso è più complesso perché richiede la presenza di tutte le classi collegate.

Obiettivo: mantenere un **accoppiamento basso**, così da avere classi più indipendenti, comprensibili e riutilizzabili.

High Cohesion

Problema:

Come mantenere le classi focalizzate, comprensibili e gestibili (favorendo anche il *Low Coupling*)?

Concetto:

La **coesione** misura quanto le responsabilità di una classe siano correlate e concentrate dal punto di vista funzionale.

- Una classe è coesa se le sue definizioni e operazioni sono strettamente legate fra loro.
- Una classe con **bassa coesione** svolge troppe attività non correlate → diventa difficile da comprendere, mantenere e riusare.

Soluzione:

Assegnare responsabilità in modo che la coesione rimanga alta.

Questo rende la classe più semplice da:

- capire,
- mantenere,

- riutilizzare,
- e meno soggetta a cambiamenti continui.

Controller

Problema:

Qual è il primo oggetto oltre l'interfaccia utente (UI) che riceve e coordina un'**operazione di sistema**?

Concetto:

Un **controller** è l'oggetto che, subito dopo la UI, riceve gli eventi principali di input del sistema e gestisce i messaggi corrispondenti alle operazioni di sistema.

Esempio: in un word processor, il comando "controllo ortografico" genera un evento di sistema che il controller riceve per avviarne l'esecuzione.

Soluzione:

Assegnare la responsabilità del controller a una classe che:

- rappresenta un dispositivo o un punto di accesso al sistema/sottosistema,
- oppure rappresenta uno scenario di caso d'uso in cui avviene l'evento di sistema.

Spesso prende nomi come <UseCaseName>Handler, <UseCaseName>Coordinator o <UseCaseName>Session.

In una sessione, lo stesso controller gestisce tutti gli eventi relativi allo stesso caso d'uso.

Il **Controller** è un pattern di **delega**: lo strato UI non deve contenere logica applicativa, ma deve **delegare** il lavoro a un altro strato.

Questo "altro strato" è quello del **dominio**, in cui l'oggetto designato prende il nome di **oggetto controller**.

Il controller riceve le richieste di lavoro provenienti dalla UI e le instrada verso gli oggetti appropriati.

Legge di Demetra (Principle of Least Knowledge)

- Non è un vero e proprio **pattern GRASP**, ma una **linea guida** fondamentale nello sviluppo software orientato agli oggetti.
- Principio generale:
 - Ogni unità di programma dovrebbe conoscere **solo poche altre unità strettamente correlate**.
 - Ogni unità dovrebbe interagire **solo con oggetti che conosce direttamente**.

Questo si riassume nel motto: **“Non parlate con gli sconosciuti”**.

